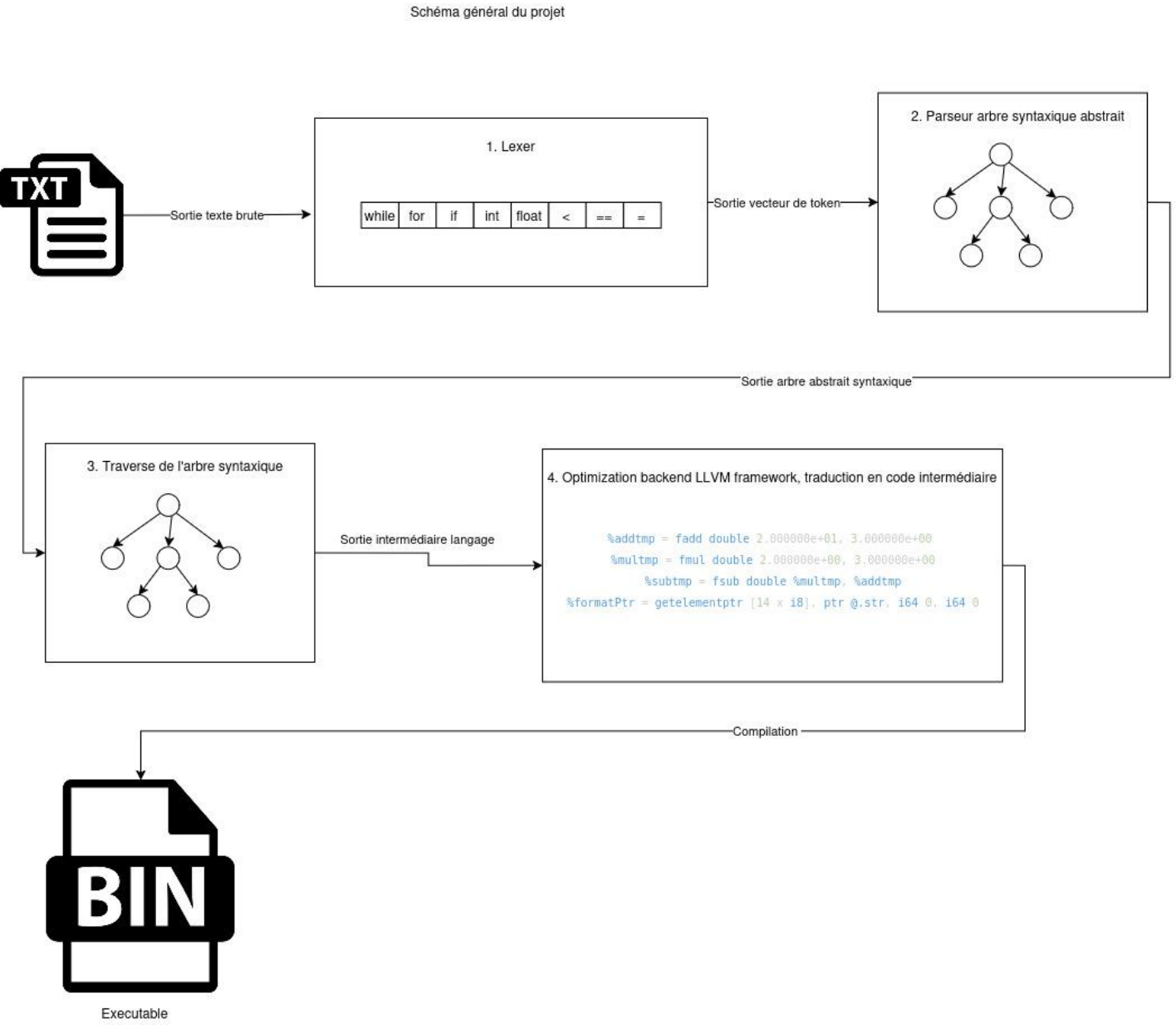


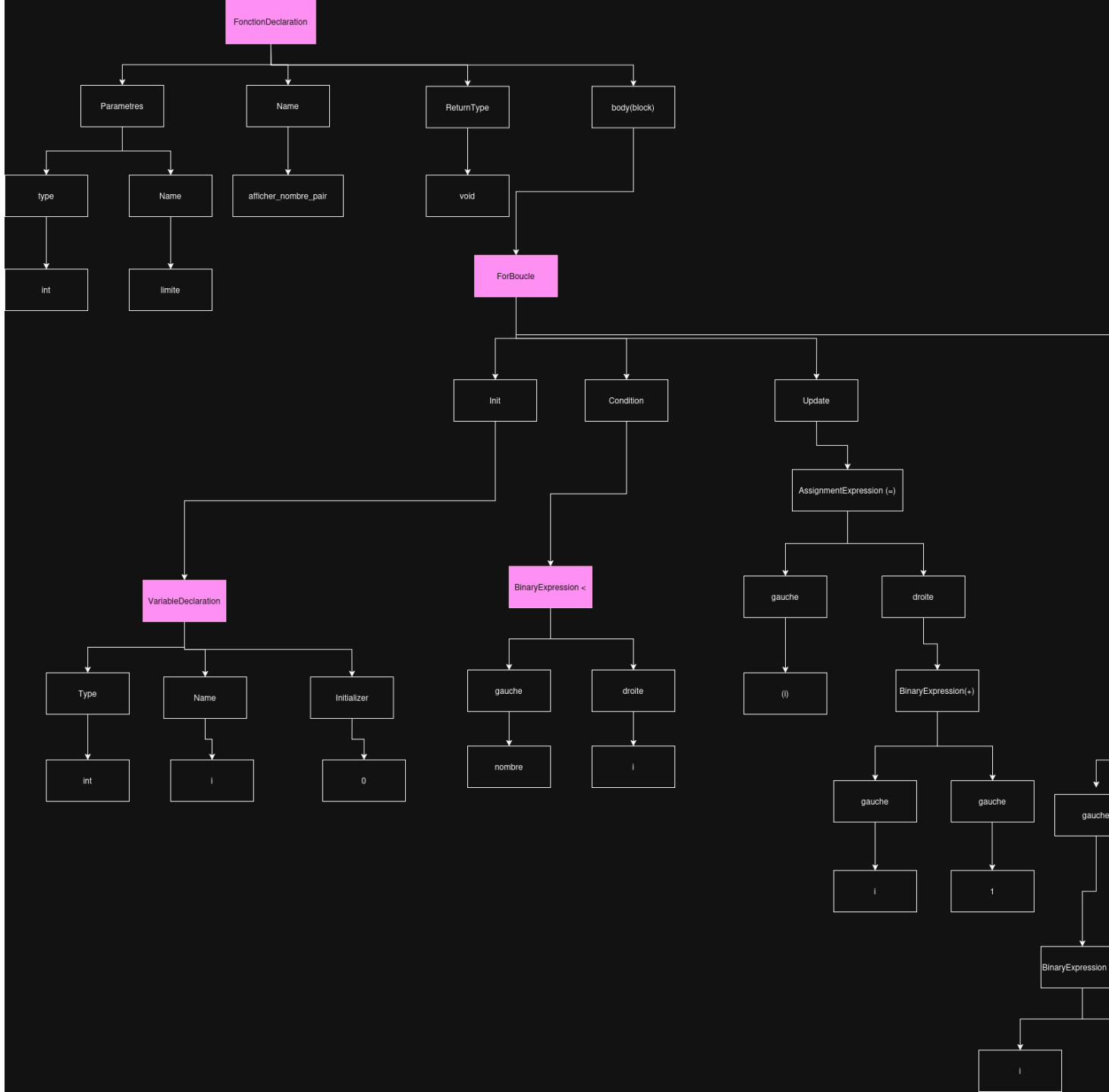

# Vue d’ensemble du projet

# Sprint 1

- - Lire des fichiers sources Prysma et les traiter comme du texte
- - Supporter les types entiers (int), flottants (float), booléens (bool)
- - Compiler les opérations arithmétiques de base (+, -, *, /) vers du code LLVM
- - Permettre au développeur de déclarer des variables et les utiliser dans des expressions

- - Valider la syntaxe du code et rejeter les structures invalides via la construction de l'AST
- - Supporter les fonctions, récursive et un type de retour.

- - Signaler les erreurs de syntaxe précisément (numéro de ligne, type d'erreur) plutôt que planter silencieusement
- - Ajouter la coloration syntaxique du langage Prysma dans Visual Studio Code



# Sprint 2

- Exporter l'AST en format texte et graphique (Graphviz) pour faciliter le débogage

- Supporter l'instruction if,

- Supporter l'instruction while, passer par la pile avec alloca

- Supporter l'approche orientée objet avec des classes, polymorphisme fait avec une vtable.

# Sprint 3

- Analyse syntaxique abstraite pour la logique du langage Prysma, ce qui évitera les problèmes de logique sémantique exemple : declare int a = "string";

- Supporter les tableaux programmés en LLVM IR

- Supporter les types string, j'utilise la bibliothèque native du C avec un extern

- Écrire une bibliothèque de structures de données (listes chaînées, arbres binaires, piles, files)

# Choix technologiques effectués

- **Backend : LLVM 18 & C++** J’ai choisi de développer le compilateur en C++ en m'appuyant sur l'infrastructure LLVM 18. C’est une version moderne qui impose les Opaque Pointers (ptr), ce qui rend la génération de code plus flexible. L’objectif est de produire un code intermédiaire (IR) performant et portable, capable d'être optimisé directement par les outils standards de l'industrie. C'est de l'assembleur portable : nous n'avons plus les contraintes de l'assembleur classique, comme la gestion des registres physiques, car ils sont virtuels (en nombre infini dans LLVM). En assembleur direct, je devrais gérer moi-même l'allocation des registres et les transferts entre la RAM et le processeur pour optimiser la vitesse, ce qui représente des algorithmes extrêmement complexes et un travail à part entière. Avec LLVM, je bénéficie de plus de 20 ans de recherche en optimisation.

# Défis rencontrés

- **La grosse refacto du Visiteur :** C'était devenu n'importe quoi, je passais 4 heures à injecter des objets partout dans les constructeurs des nœuds, c'était trop long et fastidieux, j'en ai payé les frais. J'ai tout basculé sur un système de visiteur avec une boîte à outils (`ContextGenCode`). C'est beaucoup plus propre pour le futur et ça m'évite de modifier 30 fichiers pour une seule fonctionnalité.
- **Le combat avec LLVM et les types :** Je me suis pris une vieille erreur de segmentation à cause d'un float 64-bit que j'essayais d'entrer dans un 32-bit. J'ai dû coder un système de cast pour que ça passe. J'ai aussi dégagé les dépendances LLVM de mes nœuds pour que l'architecture soit plus saine et que je ne dépende plus du backend pour résoudre mes équations. Le problème, c'est qu'un float de 64-bit est de 8 octects alors que celui de 32-bit est de 4 octects, on pouvait le voir dans le code LLVM généré, c'était écrit align 8 et j'essayais d'entrer la valeur dans un align 4 octects. L'erreur était un segmentation fault c'était invisible, j'ai dû prendre du temps pour comprendre LLVM plus en profondeur, rechercher les techniques de debuguage conventionnel, utiliser -- sanitanize  de clang pour avoir plus d'information, regarder des exemples de code LLVM.

# Défis rencontrés

- **Fonctions et variables :** J'ai enfin une pile de contexte pour avoir des variables privées par fonction. J'ai implémenté le passage d'arguments et le retour dynamique (`def int`). Par contre, j'ai toujours un bug sur les appels `call(1,1)`, seules les variables passent pour l'instant, les chiffres bruts bloquent encore, je vais devoir corriger ça plus tard.
- **Débogage et confort :** J'ai travaillé sur le lexer pour avoir les numéros de ligne et de colonne. Maintenant j'ai les vagues rouges directement dans VSCode, avant lors des tests, j'avais une imprécision et je ne savais pas si c'était la logique du consommateur de token ou mon propre code écrit en Prysma qui n'était pas correcte, écrire def au lieu de dec sans le savoir.

# Défis rencontrés

- **La documentation obscure de LLVM** (https://llvm.org/) est conçue pour les experts. La navigation y est difficile et chaque version de LLVM a des fonctionnalités et un fonctionnement différents. J'utilise la version 18 ; les différences majeures entre les versions complexifient la recherche et le débogage.
- Les IA généralisent beaucoup et peuvent donner des exemples de code provenant de LLVM 14 qui ne fonctionnent pas pour mon projet actuel. J'ai eu quelques problèmes à cause de ça. Une technique que je vais mettre en place à l'avenir consiste à compiler du code C++ pour en extraire le code LLVM généré exactement par la version 18. Je pourrai ensuite demander à l'IA de se baser spécifiquement sur ce code et sur la documentation de cette version afin d'éviter les hallucinations. C'est une piste solide pour le futur. Ça va me permetre de mieux comprendre les subtilités de LLVM en évitant des fausses informations qui me font perdre du temps.

# Ce qui reste à développer pour la prochaine itération

- L'ittération 1 est terminée car le coeur du langage est là. Par contre, je ne peux pas dire que tout est fini. Je dois permettre le stockage du return d'une fonction dans une variable, ce n'est pas encore fait, c'est un problème que je vais résoudre dans l'ittération 2, et les petits problème au niveau du passage de nombre dans une fonction je dois programmer ça aussi.